



Lista de Compras: Workstation para Agentes IA

➤ Gigabyte B650 Gaming X AX V2 — Plataforma AM5, 100 % vía APIs (sin GPU dedicada)

Autor: Prof. CERO

Fecha: 12 de septiembre de 2026

Índice

1. Resumen	2
2. Lista de Componentes	2
3. Detalle y Compras	3
3.1. Motherboard — Gigabyte B650 Gaming X AX V2	3
3.2. CPU — AMD Ryzen 7 7700X	3
3.3. RAM — 64 GB DDR5 (2 paquetes de 32 GB)	3
3.4. Almacenamiento — Doble NVMe	3
3.5. Fuente de Poder — 650 W 80 Plus	4
3.6. Disipador — Torre de aire eficiente	4
3.7. Gabinete — Frontal Mesh	4
4. Qué NO comprar	4
5. Ejemplos de uso	5
5.1. Ejemplo 1: Verificación de compilación LaTeX	5
5.2. Ejemplo 2: Enrutamiento multi-LLM determinista	5
5.3. Ejemplo 3: Generación de material académico	5
5.4. Ejemplo 4: Reparación de LaTeX con \text{}	6
5.5. Ejemplo 5: RAG sobre los documentos del workspace	6
5.6. Ejemplo 6: Control de versiones semántico	6
5.7. Ejemplo 7: Vigilancia del saldo OpenRouter	6
5.8. Ejemplo 8: Higiene del estado de sesión	7
6. Veredicto	7

1

 Resumen

Esta es la lista de compras completa para ensamblar una **estación de trabajo dedicada al desarrollo de Agentes de IA**, orientada a un flujo **100 % basado en APIs** (OpenRouter, Gemini, Groq), sin necesidad de GPU dedicada para inferencia local.

La base es la placa **Gigabyte B650 Gaming X AX V2**, sobre la plataforma **AM5** de AMD. El presupuesto de la GPU se redistribuye en **más RAM y almacenamiento**, que son los cuellos de botella reales de un flujo de desarrollo de agentes (ChromaDB, embeddings, múltiples procesos Python, servidores MCP y orquestación en paralelo).

 **Concepto de la build**

El cómputo pesado (inferencia de modelos) lo realiza el LLM remoto. Esta máquina es la **orquestadora**: corre Python, LangChain, ChromaDB, Docker, LaTeX y múltiples terminales en paralelo sin fricción.

2

 Lista de Componentes

Componente	Modelo recomendado	Precio aprox.	Link
Motherboard	Gigabyte B650 Gaming X AX V2 (ATX, AM5, WiFi 6E, 2.5G LAN)	\$160	3.1
CPU	AMD Ryzen 7 7700X (8C/16T, Zen 4)	\$280	3.2
RAM	G.Skill Flare X5 32 GB (2×16 GB) DDR5-5600 – x2 para 64 GB	\$180	3.3
SSD 1 (OS)	WD Black SN770 1 TB NVMe PCIe 4.0	\$80	3.4
SSD 2 (Datos)	WD Black SN770 2 TB NVMe PCIe 4.0	\$120	3.4
PSU	Corsair CX650 650 W 80 Plus Bronze	\$60	3.5
Cooler	Thermalright Peerless Assassin 120 SE	\$35	3.6
Case	Fractal Design Meshify C o similar con frontal mesh	\$80	3.7

 **Costo total estimado**

\$995 USD

Aprox., sin GPU dedicada (mercado actual)

3

 Detalle de Cada Componente con Enlaces de Compra

3.1 Motherboard — Gigabyte B650 Gaming X AX V2

Placa base

Elección central. Plataforma AM5, formato ATX, WiFi 6E, LAN 2.5 GbE, 2× M.2 NVMe PCIe 4.0 y VRM de 12+2 fases. Suficiente para una estación de trabajo sin tarjeta gráfica dedicada.

Compra: <https://www.amazon.com/s?k=Gigabyte+B650+Gaming+X+AX+V2> Alternativa: <https://pcpartpicker.com/search/?q=B650+Gaming+X+AX+V2>

3.2 CPU — AMD Ryzen 7 7700X

Procesador

8 núcleos / 16 hilos, Zen 4, TDP de 105 W. Potencia suficiente para orquestar agentes, compilar LaTeX y correr Docker y servidores MCP en paralelo. Si el presupuesto ajusta, el **Ryzen 7 7700** (65 W) es una opción más eficiente.

Compra: <https://www.amazon.com/s?k=AMD+Ryzen+7+7700X> Alternativa: <https://pcpartpicker.com/search/?q=Ryzen+7+7700X>

3.3 RAM — 64 GB DDR5 (2 paquetes de 32 GB)

Memoria

64 GB (4×16 GB o 2×32 GB) DDR5-5600 optimizada para AMD (EXPO). Se recomienda un kit de **2×32 GB** para dejar libertad de ampliación a 128 GB. Crítica para ChromaDB, embeddings y múltiples procesos.

Compra: <https://www.amazon.com/s?k=DDR5+5600+AMD+EXPO+32GB+kit> Alternativa: <https://pcpartpicker.com/products/memory/>

3.4 Almacenamiento — Doble NVMe

SSD

SSD 1: 1 TB NVMe PCIe 4.0 para el sistema operativo, aplicaciones y entornos conda. **SSD 2:** 2 TB NVMe PCIe 4.0 para proyectos, datasets y la base 'chroma_db'. Separarlos evita colisiones de E/S.

Compra (1 TB): <https://www.amazon.com/s?k=WD+Black+SN770+1TB> Compra (2 TB): <https://www.amazon.com/s?k=WD+Black+SN770+2TB>

3.5 Fuente de Poder — 650 W 80 Plus

 PSU

650 W 80 Plus Bronze es más que suficiente sin GPU dedicada. Deja margen si más adelante se añade una aceleradora de gama media (hasta 250 W).

Compra: <https://www.amazon.com/s?k=Corsair+CX650> Alternativa: <https://pcpartpicker.com/products/power-supply/>

3.6 Disipador — Torre de aire eficiente

 Cooler

Una torre de aire de doble ventilador (tipo **Thermalright Peerless Assassin 120 SE**) rinde mejor que el disipador de stock para cargas sostenidas de compilación, manteniendo el Ryzen 7 fresco y silencioso.

Compra: <https://www.amazon.com/s?k=Thermalright+Peerless+Assassin+120+SE>

3.7 Gabinete — Frontal Mesh

 Case

Formato ATX con frontal de malla perforada y buen flujo de aire. No se necesita un gabinete exótico; un chasis mesh medio basta para disipar la baja carga térmica de esta build.

Compra: <https://www.amazon.com/s?k=ATX+mesh+case> Alternativa: <https://pcpartpicker.com/products/case/>

4 Qué NO hace falta comprar

 Ahorros conscientes

- **GPU dedicada:** No es necesaria para un flujo 100 % API. Ahorro de \$300–\$800.
- **PSU de alta potencia (850 W+):** Sobra. Ahorro de \$40+.
- **Enfriamiento líquido:** Innecesario a 105 W. Ahorro de \$80+.
- **PCIe 5.0 en GPU:** B650 no lo ofrece y no se necesita para orquestación.

5 Ejemplos de Uso del Workspace

Los siguientes ejemplos muestran cómo esta estación de trabajo ejecuta el flujo real del workspace ELECTRONICA, siempre bajo la arquitectura de 3 capas: directiva (qué hacer) → orquestación (decidir cuál) → script determinista (cómo). Todos los comandos se ejecutan desde la raíz del repositorio.

5.1 Ejemplo 1: Verificación de compilación LaTeX

Genera el PDF con el compilador del proyecto y valida que el resultado no tenga errores, overfull ni solapamientos:

```
1 python3 execution/compile_latex.py docs/PC_PARA_IA/  
  PC_IA_B650_WORKSTATION.tex --clean  
2 python3 execution/verificar_pdf.py --pdf PC_IA_B650_WORKSTATION.pdf
```

</> Ejemplo

Resultado esperado. La compilación termina en exit 0 y el verificador reporta 0 errores / 0 overfull / 0 solapes. El artefacto se publica en docs/PC_PARA_IA/.

5.2 Ejemplo 2: Enrutamiento multi-LLM determinista

La elección del modelo no la hace el agente “a ojo”: la decide enrutador.py con reglas puras (tipo de tarea, tokens medidos, criticidad):

```
1 python3 execution/enrutador.py --task examen_complejo --tokens 12000 --  
  critico
```

</> Ejemplo

Salida tipo. Devuelve un JSON {tier, model, fallback} (p. ej. opus → anthropic/claude-opus-5). El mismo descriptor SIEMPRE produce el mismo resultado. La decisión es 100 % local y no consume créditos OpenRouter.

5.3 Ejemplo 3: Generación de material académico

Un flujo de 3 capas elabora un examen con su solucionario LaTeX a partir de un tema:

```
1 python3 flujo_elaborar_examen.py --tema "Semana 4: Condensadores" --  
  output examenes/
```

</> Ejemplo

Flujo. La elaboración consume créditos OpenRouter (tier opus). La orquestación registra el progreso en `.tmp/run_state.json` para poder retomarla tras un fallo.

5.4 Ejemplo 4: Reparación de LaTeX con `\text{}`

Script determinista que extrae comandos matemáticos de `\text{}` (errores clásicos de autogeneración):

```
1 python3 fix_latex.py documento.tex
```

5.5 Ejemplo 5: RAG sobre los documentos del workspace

Reconstruye los índices vectoriales (ChromaDB) y luego consulta el asistente RAG:

```
1 python3 rag_system.py --update      # reconstruir embeddings
2 python3 rag_system.py               # chat sobre el corpus
```

</> Ejemplo

Nota de I/O. Los embeddings residen en `chroma_db/` (excluido de Git por superar los 100 MB) y su estado vive en `db_state.json`. La memoria RAM amplia de esta build (64 GB) permite cargar el corpus completo sin fricción.

5.6 Ejemplo 6: Control de versiones semántico

El flujo git aplica versionado semántico y sincroniza con el remoto:

```
1 ./update_repo.sh -m "feat(x): mejora documentada" --push
```

</> Ejemplo

Alternativa. `./git-update.sh` es el atajo rápido: commit WIP + pull + push en un paso.

5.7 Ejemplo 7: Vigilancia del saldo OpenRouter

Chequeo puntual o bucle en segundo plano con alerta audible cerca del auto top-up de \$5 (saldo por debajo de \$3):

```
1 python3 execution/monitor_saldo_openrouter.py
2 python3 execution/monitor_saldo_openrouter.py --watch --interval 300
```

5.8 Ejemplo 8: Higiene del estado de sesión

Verifica la frescura de los estados derivados (elimina orfanatos antes de reanudar flujos multi-paso) y registra el eslabón semántico en las bitácoras:

```
1 python3 execution/estado_sesion.py check
2 python3 execution/bitacoras.py nueva --tema "<tema de la sesion>"
```

Recuerda

Regla de oro. Antes de modificar cualquier `.tex`, leer `.agent/latex.md` (16 pitfalls documentados); antes de scripts con LangChain o parsing de JSON de LLM, leer `.agent/python.md`. Esta máquina está diseñada para sostener estos flujos en paralelo sin cuellos de botella.

6 Veredicto Final

Conclusión

Esta build entrega la mejor relación rendimiento-precio para el desarrollo de Agentes de IA vía API: CPU de 8 núcleos, 64 GB de RAM DDR5 y doble NVMe, todo sobre la sólida plataforma AM5 de la **B650 Gaming X AX V2**, sin gastar un dólar de más en hardware que no se aprovecharía.

El único upgrade futuro genuino sería añadir una GPU solo si posteriormente decides correr modelos pequeños localmente.